Business Case - Stock Trading System

Executive Summary

The company needs to train its stockbrokers on how to buy and sell stocks without exposing the firm or its clients to the risk of real-money mistakes. The IT department proposes to design and develop an in-house Stock Trading System: a web application in which trainees buy and sell simulated stocks whose prices fluctuate during market hours, managed by an administrator who creates stocks and controls the market schedule. This document recommends approval to design, build, and deploy the system.

Problem Statement

New stockbrokers currently have no safe, hands-on environment in which to practice trading. Learning on live markets risks costly errors, and classroom-only training does not build practical skill in placing, confirming, and settling orders or in managing a portfolio and cash account. The company needs a realistic practice environment that behaves like a real market but carries no financial risk.

Proposed Solution

Build a 3-tier web application: a browser-based interface for trading, a Flask application server on AWS EC2 for business logic, and a MySQL database on AWS RDS for secure data storage. Customers (trainees) create accounts, deposit and withdraw simulated cash, buy and sell stocks at market price during market hours, confirm each transaction before it is fulfilled (transactions cannot be cancelled), and view their portfolio and transaction history. Administrators create stocks and set initial prices and the market schedule. A random price generator determines market prices, and the system enforces market hours and logs all account, stock, and cash events with timestamps.

Options Considered

- Option 1: Build in-house (recommended) – full control of features, realistic company-specific training, low-cost cloud hosting, and reusable IT experience.
- Option 2: Buy a commercial trading simulator – faster to deploy but recurring license fees, limited control over features and data.
- Option 3: Do nothing – continue classroom-only training; practical skill gap and error risk remain.

Benefits

- Risk-free training: brokers practice full trade lifecycles with zero financial exposure.
- Faster onboarding: hands-on practice shortens time-to-competency for new hires.
- Measurable progress: transaction history and event logs let managers review each trainee's activity.
- Low operating cost: runs on a small EC2 instance and RDS database with no per-user license fees.

Costs (Estimated)

One-time development by the internal IT team (design, build, test); ongoing AWS hosting for one EC2 instance and one RDS MySQL instance; minor maintenance effort after launch. No commercial software licenses are required.

Risks and Mitigation

- Security exposure of a public web application – mitigated by isolating the database in a private subnet and logging all events.
- Unrealistic price behavior reduces training value – mitigated by a gradual, market-hours-only price generator.
- Schedule slippage – mitigated by phased delivery: accounts and cash first, then trading, then administration.

Recommendation

Approve Option 1 - the design and development of the Stock Trading System as specified in the accompanying requirements document. The system addresses a clear training need at low cost, with manageable risk and full ownership of the solution.

The project is to create a stock trading system where users can buy and sell stocks. The system will support two types of users. The first is the customer, who will buy and sell stocks, and the second is the administrator of the system, who is responsible for creating the stocks and setting the initial price.

Business Requirements

Customer Required Functions:

- Buy and sell stocks at market price when the market is open.
    - Stock purchases will be paid for from the user's cash account.
    - The user will receive a confirmation before the transaction is fulfilled.
    - There is no cancellation of transactions.
    - The system will validate that sufficient funds are available before a purchase.
    - When stocks are sold, the shares are returned to the stock inventory.
    - Portfolio, order history, and financial records are updated after each transaction.
- View their current portfolio of stocks and cash.
- View their history of transactions.
- Deposit and withdraw cash.
    - When the user deposits cash, the funds will go into their cash account.
    - The user will only be able to withdraw money from their cash account.
    - When stocks are sold, the funds will go into the cash account.

Administrators Functions:

- Maintain the stock inventory that can be purchased by users.
- Create new stocks.
    - Include the company name, stock ticker, amount of stock (shares outstanding), and initial price.
    - The administrator sets the initial price only; after creation, the stock's price is determined by the market.
- Change market hours.
    - Market schedule: by default, the market will be open on weekdays from 9:00 AM to 5:00 PM and closed on holidays.

System Functions:

- Random Stock Price Generator: determines the market price of each stock. Prices fluctuate gradually up or down during market hours and do not change while the market is closed.
- Existing stocks always display the current market price.
- Market hours enforcement: users can only execute trades while the market is open, per the schedule set by the administrator.
- Order execution: orders are executed by the system during market hours after the user confirms the transaction; once confirmed, a transaction cannot be cancelled. Proceeds from stock sales are automatically deposited into the user's cash account.
- Event logging: the system records timestamps for account creation and for all stock and cash transactions.

Technical Requirements: Design & Develop a Web Application

Note on Architecture: This system is a web-based client/server application. "Client/server" describes the architectural model, in which a client makes requests and a server responds, while "web application" describes the specific implementation. In this system, the client is the user's web browser (Tier 1), which renders the HTML, CSS, and JavaScript, and the server is the Flask application running on EC2 (Tier 2), which processes HTTP requests, applies business logic, and communicates with the MySQL database (Tier 3). The browser never accesses the database directly. Therefore, this web application is implemented using a 3-tier architecture.

1. Graphical User Interface (web pages):
    - Display available stocks that can be traded.
    - Perform the user and administrator functions.

2. Relational Database at the backend, which stores the data. The database contains user data and system data. System data is used for calculations and for security (i.e., logging events). For example, timestamps for account creation and timestamps for stock or cash transactions.

3. The system must be fully deployed and operated within the AWS Cloud environment.

3-tier Logical Architecture Structure

Tier 1: Presentation Tier

- Location: User's browser or client
- Function: Focused on user interface and experience
- Components: HTML templates, CSS, and JavaScript served by Flask
- Access: Designed for public internet users to interact with directly

Tier 2: Application Tier (Business Logic)

- Location: EC2 instance in a public subnet
- Function: Handles business logic, processes requests, and manages application logic
- Components: Flask web framework, Python code, and API endpoints
- Port: Exposed on 8000 to the internet
- Responsibilities: Includes handling HTTP requests, processing business rules, communicating with the database, and generating responses

Tier 3: Data Tier

- Location: MySQL database on an RDS instance within a private subnet
- Function: Handles data storage and management
- Components: MySQL database
- Port: Accessible only from the application tier on 3306
- Security: Fully isolated in the private subnet with no direct internet access